AI Sales Forecasting and Intelligent Business Decision Support System

Balamurugan R 241801031
Department of Computer Science and Engineering
Rajalakshmi Engineering College, Tamil Nadu, India
Email: balamurugan.r.2024.aids@rajalakshmi.edu.in

Abstract—Retail and e-commerce businesses accumulate large volumes of transactional data, yet many still rely on manual spreadsheet analysis that offers limited forecasting and no support for scenario planning. This paper presents an AI-based sales forecasting and business decision support web application. Users upload CSV or Excel sales records containing date, quantity, price, discount, and advertising expenditure. The system validates and cleans the data, engineers calendar and lag features, and trains Random Forest and Gradient Boosting regressors that are combined in a weighted ensemble, with an ARIMA model serving as a time-series reference. The platform is implemented with a FastAPI backend, a browser-based dashboard, and optional MongoDB persistence, and it also provides robust IQR-based anomaly detection, what-if scenario simulation, and an optional AI chat assistant. On a 1,000-record retail dataset evaluated with a chronological 80/20 split, the ensemble achieved an MAE of 2.87, an RMSE of 3.79, and an R^2 of 0.838, reducing MAE by 75.2% relative to a last-observation baseline. The results suggest that the system can give small businesses an accessible, validated forecasting workflow, although its scenario outputs are associative rather than causal.

Keywords—Sales Forecasting, Machine Learning, Retail Analytics, Random Forest, Gradient Boosting, ARIMA, Decision Support, FastAPI.

I. INTRODUCTION

Sales forecasting is a central activity in retail and e-commerce because estimates of future demand drive inventory purchasing, staffing, pricing, and promotional planning. Classical statistical models such as ARIMA [1] remain widely used, while ensembles of decision trees, namely Random Forests [2] and Gradient Boosting [3], have become strong general-purpose learners for tabular business data. Large forecasting competitions further indicate that combining several methods tends to be more reliable than relying on a single one [4].

Despite this progress, many small and medium-sized businesses still analyze sales manually in spreadsheets. Such workflows involve repetitive cleaning, provide little or no validated forecasting, and offer no way to explore how pricing, discounts, or advertising might change demand. Moreover, a forecast that is not compared with a simple baseline on data held out in time can appear more accurate than it really is [5]. There is therefore a need for an accessible tool that couples data preparation, honest validation, and scenario exploration in a single web interface.

This paper presents an AI Sales Forecasting and Intelligent Business Decision Support System. Its main contributions are:

(a) an upload-to-decision pipeline that accepts CSV and Excel files and performs automatic validation, cleaning, and feature engineering;
(b) a hybrid forecasting engine that combines Random Forest and Gradient Boosting regressors with an ARIMA reference model;
(c) a leakage-aware chronological evaluation that reports MAE, RMSE, R^2, and skill against a last-observation baseline [6];
(d) decision-support modules for permutation-based driver analysis, robust anomaly detection, and what-if scenario simulation; and
(e) a deployable FastAPI web application with optional MongoDB persistence and an optional AI chat assistant.

The remainder of this paper is organized as follows. Section II reviews related work, Sections III and IV describe the proposed system and its architecture, Section V details the methodology, Section VI covers implementation, Section VII presents results, Section VIII discusses limitations and future work, and Section IX concludes.

II. RELATED WORK

A. Statistical Time-Series Forecasting

ARIMA models describe a series through autoregressive, differencing, and moving-average terms and are a standard reference for univariate forecasting [1]. Their strengths are parsimony and interpretability; their limitation is that they depend only on the series' own history and cannot directly use external drivers such as price or promotion. Implementations are available in the statsmodels library [7].

B. Machine-Learning Forecasting and Ensembles

Random Forests average many decorrelated decision trees and are robust to noise and nonlinearity [2]. Gradient boosting builds trees sequentially, each correcting the residual errors of the ensemble so far [3]. Ensemble learning in general improves accuracy and stability by combining diverse models [8], and the M4 competition reported that combination approaches were among the strongest performers [4]. The proposed system adopts this principle by blending two tree ensembles with an ARIMA component.

C. Validation and Interpretability

Time-ordered data violates the independence assumption behind random train/test splits, so forecasts should be evaluated out of sample in chronological order [5], [9]. Errors should be reported with appropriate accuracy measures and compared against naïve benchmarks [6]. For explanation, permutation importance measures how much prediction error rises when a feature is shuffled [10]; it reflects predictive reliance rather than causal effect.

D. Anomaly Detection

Anomaly detection identifies observations that deviate from expected behavior [11]. Simple robust rules based on

the interquartile range [12] remain attractive in decision-support tools because they are transparent and require no training, which lets analysts understand why an observation was flagged.

E. Research Gap

Studies centered on model accuracy often stop at reporting error figures. A practical decision-support tool additionally needs data validation, honest baselines, driver analysis, anomaly review, and scenario simulation delivered through an interface that non-specialists can use. This is the gap the present work addresses.

III. PROPOSED SYSTEM

A. Overview

The proposed system is a web application that takes a business user from a raw sales file to forecasts and decision support without writing code. A user uploads a CSV or Excel file; the system cleans the data, trains the models automatically, and presents forecasts, validation metrics, anomalies, recommendations, and a scenario simulator on a dashboard.

B. Functional Modules

Data ingestion. Files of type .csv, .xlsx, or .xls up to 10 MB and 50,000 cleaned rows are accepted. *Date* and *Quantity* are required; *Price*, *Discount*, *Advertising*, *Product*, *Season*, and *Region* are optional. Common aliases such as "Order Date" or "Units Sold" are mapped automatically. Training requires at least 10 valid rows and 5 distinct dates.

Forecasting engine. Random Forest and Gradient Boosting regressors are blended and combined with an ARIMA reference model (Table I).

TABLE I
ALGORITHMS AND SETTINGS USED IN THE SYSTEM

Algorithm	Role in system	Key settings
Random Forest	Regressor (weight 0.65)	250 trees, depth 10
Gradient Boosting	Regressor (weight 0.35)	150 stages, depth 3, rate 0.05
ARIMA	Time-series reference	Order (1,1,1)
Permutation importance	Driver explanation	5 repeats on holdout
IQR rule	Anomaly detection	1.5 × IQR fences

Evaluation and explanation. A chronological holdout yields MAE, RMSE, R^2, and baseline skill. Permutation importance and rule-generated recommendations state their evidence and confidence level.

Anomaly detection and what-if analysis. Unusual records are queued for review, and up to five scenarios can be simulated over horizons of up to 30 periods.

AI assistant. An optional chat panel answers questions grounded in the current dashboard context using a Gemini or Claude model.

IV. SYSTEM ARCHITECTURE

The architecture follows a three-tier design, shown in Fig. 1. The presentation tier is a browser dashboard written in HTML, CSS, and JavaScript with five views (Dashboard, Data Pipeline, AI Recommendations, What-If, and Anomalies) and a floating chat panel. It communicates with the application tier through JSON REST endpoints (/api/upload, /api/train, /api/dashboard, /api/whatif, /api/chat, and /api/health).

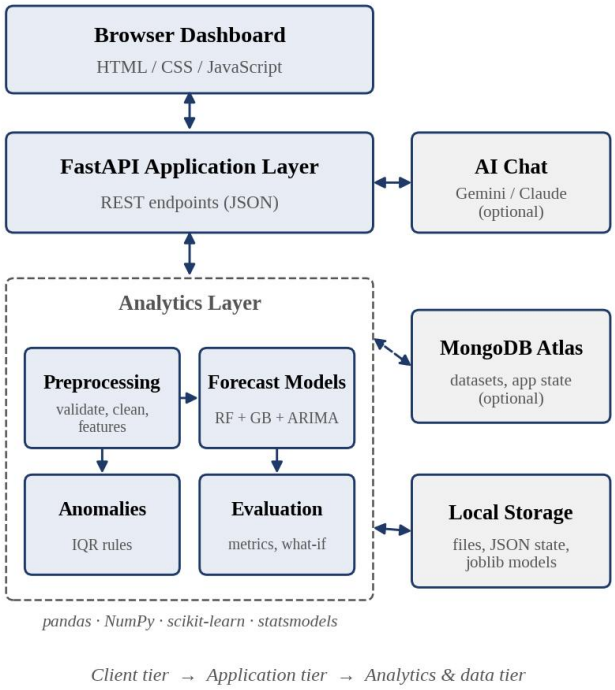


Fig. 1. System architecture of the proposed AI sales forecasting system.

The FastAPI application tier validates requests and invokes the analytics tier, which is built on pandas, NumPy, scikit-learn, and statsmodels. Persistence is local by default (uploaded files, a JSON state file, and joblib model artifacts); when a MongoDB connection string is configured, cleaned datasets and application state are additionally stored in MongoDB Atlas. The optional chat assistant calls the Gemini or Claude API from the server so that API keys are never exposed to the browser.

The end-to-end workflow is illustrated in Fig. 2. After upload, the data are validated and cleaned, features are engineered, and the models are trained on the earlier portion of the timeline. Forecasts, metrics, and explanations are then served to the dashboard, from which the user can review anomalies and run what-if analysis.

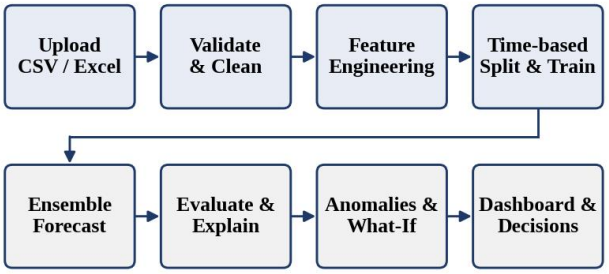


Fig. 2. Workflow of the sales forecasting and decision-support pipeline.

V. METHODOLOGY

A. Data Collection and Validation

Records are read with pandas from CSV or Excel files. Column names are normalized for case and spacing and mapped through an alias table. Files lacking a date or

quantity column are rejected with an explanatory message, and file type and size are checked before parsing.

B. Data Preprocessing

Dates are parsed and unparsable rows are dropped, and numeric fields are coerced to numbers. Missing *Price*, *Discount*, and *Advertising* values are replaced by the column median (or by defaults of 1, 0, and 0 when a column is absent). Rows with non-positive price, discount outside 0–100, negative advertising, or negative quantity are removed; exact duplicates are dropped; and the data are sorted chronologically. Absent categorical fields default to "All Products," "Unknown," and "All Regions."

C. Feature Engineering

Ten predictors are used: Price, Discount, Advertising, Day, Month, Year, DayOfWeek, Lag1, Lag7, and Rolling7. Lag1 and Lag7 are the quantities one and seven observations earlier. To prevent target leakage, Rolling7 is the mean of the seven observations that precede the current one:

$$\text{Rolling7}_t = (q_{t-1} + q_{t-2} + \ldots + q_{t-7}) / 7 \quad (1)$$

At the start of the series, missing values are filled with the median or a running mean.

D. Chronological Split

The unique dates are sorted, the earliest 80% form the training set, and all later dates form the test set. Rows that share a date are kept together, so no date appears in both sets, and the models are fitted only on training rows.

E. Forecasting Models

A Random Forest regressor and a Gradient Boosting regressor (Table I) are trained with a fixed random seed, and their predictions are blended as

$$\hat{y}_{ML} = 0.65\ \hat{y}_{RF} + 0.35\ \hat{y}_{GB} \quad (2)$$

Multi-step forecasts are produced recursively: each prediction is fed back into Lag1, Lag7, and Rolling7 for the next step, future dates are inferred from the dataset's observed cadence, and Price, Discount, and Advertising are held at their latest values. An ARIMA(1,1,1) model [1] is fitted to the quantity series as a time-series reference, with a linear-trend fallback if fitting fails. The final dashboard forecast is

$$\hat{y}_{final} = 0.75\ \hat{y}_{ML} + 0.25\ \hat{y}_{ARIMA} \quad (3)$$

The blending weights are fixed design choices rather than tuned values.

F. Evaluation Metrics

Holdout accuracy is measured by the mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2), where y, $\hat{y}$, and $\bar{y}$ denote the actual, predicted, and mean values over n test rows:

$$\text{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i| \quad (4)$$

$$\text{RMSE} = \sqrt{[\ (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]} \quad (5)$$

$$R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2 / \Sigma\ (y_i - \bar{y})^2 \quad (6)$$

As a benchmark, a last-observation baseline predicts the final training quantity for every test row, and forecast skill is reported as

$$\text{Skill} = (1 - \text{MAE}_{model} / \text{MAE}_{base}) \times 100\% \quad (7)$$

G. Explainability

Permutation importance [10] is computed on the Random Forest using the holdout rows: each feature is shuffled five times and the mean increase in error is recorded. The interface presents this as predictive reliance, not causal effect.

H. Anomaly Detection

For Quantity, Price, Discount, and Advertising, an observation is flagged when it falls outside the fences [12]

$$[\ Q1 - 1.5 \cdot IQR\ ,\ Q3 + 1.5 \cdot IQR\ ] \quad (8)$$

Severity is expressed as a robust score $|x - \text{median}| / (IQR/1.349)$; scores of at least 5 are labeled "High" and others "Review." If the IQR is zero, a median-based fallback with a threshold of 3.5 is used, and the 30 highest-scoring findings are retained for review.

I. What-If Simulation

The baseline scenario uses the median historical Price, Discount, and Advertising. For each user-defined scenario, the trained ensemble is projected recursively over the requested horizon with the scenario inputs held constant. Gross revenue is estimated as

$$\text{Revenue} = \Sigma_h\ q_h \cdot P \cdot (1 - D/100) \quad (9)$$

where q is the predicted quantity in each period h, P is the price, and D is the discount percentage. Scenarios whose inputs fall outside the observed training range are labeled as extrapolation. ARIMA is shown only as an unchanged reference because it cannot respond to these inputs.

VI. IMPLEMENTATION

The backend is written in Python and served with Uvicorn. FastAPI provides the REST interface and automatic API documentation; pandas 2.2 and NumPy 2.2 handle data preparation; scikit-learn 1.6 [13] supplies the Random Forest, Gradient Boosting, metric, and permutation-importance routines; and statsmodels 0.14 [7] supplies ARIMA. Excel files are read through openpyxl. Trained models are serialized with joblib and reloaded for what-if requests, and they are retrained automatically if the artifact is missing. Optional persistence uses PyMongo to store cleaned dataset rows and the current application state in MongoDB Atlas, with local files as a fallback.

The frontend is a single-page dashboard in HTML, CSS, and JavaScript served by the same FastAPI process, so deployment requires only one service, and the repository includes a Render configuration for cloud hosting. For the chat assistant, the server sends a compact context (dataset summary, metrics, the first forecast steps, top features, anomalies, and recommendations) with the last two conversation turns to the configured model, caps the output length, and applies an 11-second timeout so that the dashboard stays responsive.

Basic safeguards include file-type and size checks, a 50,000-row limit, validation of scenario parameters, and server-side storage of API keys. The current build has no user authentication and is intended for demonstration and single-tenant use.

VII. RESULTS AND DISCUSSION

A. Dataset and Experimental Setup

The system was evaluated on a 1,000-record demonstration dataset of daily retail transactions covering ten beauty and personal-care products across four regions (250 records per region). Quantity averages 42.65 units (standard deviation 9.79); price ranges from 260 to 1,412, discount from 9% to 25%, and advertising spend from 2,750 to 13,028. No duplicate or invalid rows were found, so all 1,000 records were retained. The first 800 records formed the training set and the last 200 the test set, using the system's own training pipeline.

B. Forecast Accuracy

Table II compares the models on the 200 held-out records. Gradient Boosting achieved the lowest error (MAE 2.62), followed by the weighted ensemble (2.87) and Random Forest (3.16). The ensemble's R^2 of 0.838 means it explains roughly 84% of the variance in unseen quantities, and its MAE is 75.2% lower than that of the last-observation baseline (11.59).

TABLE II
HOLDOUT PERFORMANCE OF THE COMPARED MODELS

Model	MAE	RMSE	R^2
Random Forest	3.16	4.09	0.811
Gradient Boosting	2.62	3.57	0.856
Weighted ensemble (0.65/0.35)	2.87	3.79	0.838
ARIMA(1,1,1)	8.79	11.34	−0.450
Training-set mean	7.62	9.84	−0.093
Last-observation baseline	11.59	14.44	−1.352

Fig. 3 shows that the ensemble tracks the actual series closely, although the sharpest peaks are under-predicted. The ensemble did not outperform Gradient Boosting alone on this dataset, which indicates that the fixed 0.65/0.35 weights are not optimal here; they were retained as a transparent default and are a candidate for tuning.

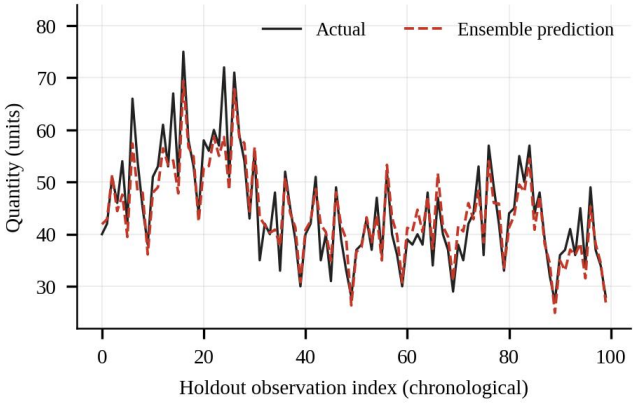


Fig. 3. Actual versus ensemble-predicted quantity on the last 100 holdout records.

The stand-alone ARIMA(1,1,1) model performed poorly (MAE 8.79, R^2 −0.450), worse than simply predicting the training mean. This is expected: the records interleave different products, so the series is not a single smooth process, and ARIMA cannot exploit price or advertising information. Because ARIMA contributes 25% of the final forecast in (3) and that blend was not evaluated separately, its weight should be validated or tuned in future work.

C. Feature Importance

Fig. 4 shows the permutation importance of the leading features. Price (0.849) and Advertising (0.287) dominate, followed by the recent-demand features Rolling7 and Lag1, while calendar features such as Month, Day, and DayOfWeek contribute almost nothing. Price must be interpreted with care: the average price differs greatly between products (from about 300 to 1,300), so Price very likely acts as a proxy for product identity rather than measuring price sensitivity.

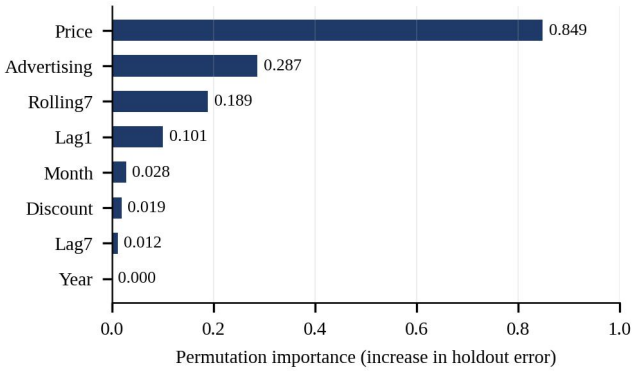


Fig. 4. Permutation importance of the leading predictors on the holdout set.

D. Anomaly Detection

The detector filled its 30-item review queue. The highest-scoring item is a Sunscreen SPF50 record dated 2026-07-15 with a quantity of 83 against a robust expected range of 18 to 66 units (score 4.72, severity "Review"). Such a spike may reflect a promotion, a seasonal event, or a data-entry error, and the interface directs the user to verify it against source records before retraining.

E. What-If Scenario Analysis

Table III shows four scenarios simulated over seven periods against a baseline at the median inputs (price 624.5, discount 16%, advertising 6,716). All scenarios lie within the observed input ranges. Raising advertising by 50% yields the largest gain (+18.5% units), whereas a deeper 25% discount leaves volume almost unchanged (−0.8%) while lowering estimated gross revenue by 11.4%.

TABLE III
SEVEN-PERIOD WHAT-IF SCENARIOS VERSUS BASELINE

Scenario	Units	Δ Units	Δ Revenue
Baseline	259.8	—	—
Discount 16% → 25%	257.8	−0.8%	−11.4%
Advertising +50%	308.0	+18.5%	+18.5%
Price +10%	268.6	+3.4%	+13.7%

The "Price +10%" result, in which predicted volume rises by 3.4% when the price rises, is economically implausible as a causal effect. It illustrates that the simulator reports what the fitted model associates with the inputs, and that Price is entangled with product identity as discussed above. Scenario results should therefore be treated as hypotheses to be verified through controlled tests, which is the guidance the interface itself displays.

F. Discussion

The results show that a lightweight tree-ensemble pipeline can forecast this dataset far better than naïve benchmarks, and that the surrounding modules turn the forecast into

actionable review items. They also show why transparent validation matters: the weaker ARIMA score, the non-optimal ensemble weights, and the implausible price scenario are all visible to the user rather than hidden behind a single headline accuracy figure.

VIII. LIMITATIONS AND FUTURE WORK

The study has several limitations. Evaluation uses one demonstration dataset and a single chronological split, which can be noisy and may not generalize to real stores; rolling-origin validation [9] would give a more reliable estimate. Records from different products are modeled together rather than per product, the ARIMA order is fixed, and the ensemble weights are not tuned. The system gives point forecasts without prediction intervals, and its scenario outputs are associative rather than causal. Finally, the build has no authentication or per-user data isolation, and model artifacts are stored locally.

Future work includes per-product and hierarchical forecasting, prediction intervals, hyperparameter and weight tuning, comparison with additional models such as XGBoost and neural sequence models, validation on real retail data, user authentication with per-user storage, and controlled experiments to estimate causal price and promotion effects.

IX. CONCLUSION

This paper presented an AI-based sales forecasting and decision-support web application that unifies data validation, feature engineering, a hybrid Random Forest, Gradient Boosting, and ARIMA forecasting engine, chronological evaluation, anomaly detection, and what-if analysis. On a 1,000-record dataset, the ensemble reached an MAE of 2.87 and an R^2 of 0.838, a 75.2% reduction in error over a last-observation baseline. By exposing its validation results and caveats, the system supports cautious, evidence-based business decisions. Planned improvements in validation, per-product modeling, and uncertainty estimation will further strengthen its practical value.

REFERENCES

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[2] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[3] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[4] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *Int. J. Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.

[5] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *Int. J. Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.

[6] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[7] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 92–96.

[8] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (MCS 2000)*, Lecture Notes in Computer Science, vol. 1857. Berlin, Germany: Springer, 2000, pp. 1–15.

[9] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, 2012.

[10] A. Altmann, L. Toloşi, O. Sander, and T. Lengauer, "Permutation importance: A corrected feature importance measure," *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, 2010.

[11] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, Art. no. 15, 2009.

[12] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[13] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.